PASCAL

Kompilator och p-kod för ABC 802

Bruksanvisning

Pascal-skivan till ABC 802 inifrån

Pascal för ABC 802. Kompilator och p-kod. Bruksanvisning

© Set Lonnert 2026

Den här bruksanvisningen är skriven 2026, i den stil som häftena till programmen hade i början av 1980-talet. Den är inte ett dokument från den tiden, och Pascal på de här skivorna är inte ett program från den tiden: kompilatorn och tolken är skrivna för boken *ABC 802 inifrån*, och de går att köra både i emulatorn och på en riktig ABC 802. Förebilderna är Niklaus Wirths Pascal-P och UCSD Pascal, som också översatte till p-kod.

Bruksanvisningen är skriven av Set Lonnert tillsammans med Claude, en AI-modell från Anthropic, använd i verktyget Claude Code. Skärmbilderna är tagna i emulatorn med Pascal-skivan och programskivan, och de tas om när programmen ändras.

Bruksanvisningen gäller Pascal som det är på skivorna i november 2026.

INNEHÅLL

Förord **5**

1 Allmänt **6**
1.1 Kompilator och p-kod 6
1.2 Två sätt att köra 6
1.3 Skivorna 6

2 Att komma i gång **8**
2.1 Starta 8
2.2 Kompilera ett exempel 8
2.3 Köra 9

3 Att skriva ett program **10**
3.1 Skriva i VERK 10
3.2 Spara 11
3.3 Kompilera och köra 11
3.4 När något är fel 12
3.5 Utan VERK 13

4 Språket **14**
4.1 Programmet 14
4.2 Tecknen på ABC 802 14
4.3 Typerna 15
4.4 Deklarationerna 15
4.5 Satserna 16
4.6 Uttrycken 16
4.7 Standardfunktionerna 17
4.8 Skriva och läsa 17
4.9 Filerna 17
4.10 Det som inte finns 18

5 Pascal under CP/M **19**

5.1 PVM.COM 19

5.2 Att skriva med SKRIV 19

5.3 Det större språket: PCS 20

6 Felmeddelanden **21**

6.1 Kompileringsfel 21

6.2 Körfel 22

6.3 Övriga 22

7 Hur Pascal fungerar **23**

7.1 Kompilatorn 23

7.2 P-koden 23

7.3 Tolken 23

8 Minneslista **25**

FÖRORD

Pascal är ett programspråk för den som vill skriva ordnade program. Där BASIC har radnummer och GOTO har Pascal procedurer med egna namn och variabler, typer som kontrolleras innan programmet körs, och satser som `WHILE` och `REPEAT` som visar hur programmet går.

Den här bruksanvisningen beskriver Pascal för ABC 802: kompilatorn `PC`, som översätter ett program till *p-kod*, och tolken som kör p-koden. Den lär dig inte Pascal från början. Kan du lite BASIC och har en lärobok i Pascal bredvid dig räcker kapitel 4 för att se vad som är med och vad som saknas.

Läs kapitel 2, *Att komma i gång*, och gör exemplet vid datorn. Kapitel 3 visar hur du skriver ett eget program med VERK från programskivan, kompilerar det och rättar felen. Kapitel 4 beskriver språket, och kapitel 5 det större språket som finns under CP/M. Kapitel 6 förklarar felmeddelandena, och längst bak finns en minneslista.

Kapitel 7, *Hur Pascal fungerar*, är för den som vill veta vad som händer inne i datorn. Det behövs inte för att skriva program.

1. ALLMÄNT

1.1 Kompilator och p-kod

BASIC i ABC 802 *tolkar* programmet: varje gång en rad körs läses den och tolkas på nytt. Pascal *kompileras*. Kompilatorn `PC` läser hela programmet en gång, kontrollerar det och översätter det. Är något fel får du veta det innan programmet har börjat köra, med rad och kolumn.

Det kompilatorn gör är inte Z80-instruktioner utan *p-kod*, koden för en tänkt maskin som är gjord för Pascal. En liten *tolk* i maskinkod kör sedan p-koden. P-koden är kort, så att stora program får plats, och den är densamma var den än körs: samma `.PK`-fil går både under BASIC och under CP/M.

Kompilatorn är själv skriven i Pascal och är själv p-kod, filen `PC.PK`. Den har kompilerat sin egen källkod, `PC.PAS`, som finns på skivan bredvid.

1.2 Två sätt att köra

I BASIC-läget sätter du Pascal-skivan i MF0 och startar datorn. Tolken laddas, och du kan kompilera och köra direkt. Programmen har omkring 24 kilobyte till p-kod och variabler. Det är sättet som beskrivs först i den här bruksanvisningen.

Under CP/M är tolken programmet `PVM.COM` på Pascal-skivan för CP/M. Där får programmen omkring 49 kilobyte, och där finns också `PCS`, en kompilator för ett större språk med poster, pekare och mängder. Se kapitel 5.

1.3 Skivorna

Pascal-skivan för BASIC-läget har

PASCAL.ABS	tolken, som laddas när datorn startar
PC.PK	kompilatorn
PC.PAS	kompilatorns källkod, i Pascal
FPP.BIN	räknerutinerna för reella tal
HALSA.PAS	en hälsning och kvadraterna 1 till 10
PRIMTAL.PAS	primtalen med Eratosthenes såll
REKURSIO.PAS	procedurer som anropar sig själva
SATSER.PAS	IF, CASE, slingorna och FORWARD
SORTERA.PAS	sortering av ett fält
LASA.PAS	läsa från tangentbordet och från en fil
REELLA.PAS	reella tal och funktionerna
STRANGAR.PAS	strängar och filnamn

Exemplen är kompilerade program att läsa och prova. Pascal-skivan för CP/M har samma exempel och kompilatorer, och dessutom PVM.COM, PCS.PK och PCS.PAS.

För att skriva egna program i BASIC-läget behövs också *programskivan*, med VERK. Den sitter i MF1.

2. ATT KOMMA I GÅNG

2.1 Starta

Sätt Pascal-skivan i MF0 och programskivan i MF1, och starta datorn. Efter ett ögonblick står det

```
Pascal för ABC 802. PC kompilerar, t.ex. PC.PAS till PC.PK.
Z=CALL(32768) kör ett program igen.

Program:
```

Tolken frågar efter ett program. Det är namnet på en `.PK`-fil, och `.PK` behöver du inte skriva. Trycker du bara RETURN kommer du till BASIC.

2.2 Kompilera ett exempel

`HALSA.PAS` är ett kort program på skivan:

```
PROGRAM HALSA(OUTPUT);
(* En hälsning och kvadraterna. *)
CONST N = 10;
VAR I, S : INTEGER;
BEGIN
  WRITELN('Hej fran Pascal!');
  S := 0;
  FOR I := 1 TO N DO
  BEGIN
    WRITE(SQR(I):4);
    S := S + I
  END;
  WRITELN;
  WRITELN('Summan 1..', N:1, ' = ', S:1)
END.
```

Kompilatorn är också ett program. Skriv `PC` vid `Program:` och tryck RETURN. Kompilatorn laddas och väntar på namnet på källan. Skriv `HALSA.PAS` och tryck RETURN.

Efter några sekunder står det `ABC802` igen. Hittade kompilatorn inget fel har den skrivit p-koden till `HALSA.PK` på skivan, och då säger den ingenting.

2.3 Köra

Tolken ligger kvar i minnet. Du startar den igen från BASIC med

```
Z=CALL(32768)
```

och svarar `HALSA` vid `Program:`:

```
Pascal för ABC 802. PC kompilerar, t.ex. PC.PAS till PC.PK.
Z=CALL(32768) kör ett program igen.

Program: PC
HALSA.PAS
ABC802
Z=CALL(32768)
Program: HALSA
Hej fran Pascal!
   1   4   9  16  25  36  49  64  81 100
Summan 1..10 = 55
ABC802
```

När programmet är slut är du tillbaka i BASIC. `Z=CALL(32768)` och ett namn kör ett program till, och `Z=CALL(32768)` och `PC` kompilerar ett till.

Prova de andra exemplen på samma sätt. `PRIMTAL` räknar ut primtalen under 8192 på några sekunder; BASIC behöver betydligt längre tid för samma sak.

Observera. Skriv filnamnen med stora bokstäver. Kompilatorn söker på skivan efter namnet precis som du skrivit det.

3. ATT SKRIVA ETT PROGRAM

Pascal-skivan har ingen egen editor. Programmet skriver du i VERK, från programskivan, och sparar det på Pascal-skivan. VERK är en ordbehandlare, men en text i VERK är bara tecken, och ett Pascalprogram är också det. Kan du ORD eller VERK sedan förut känner du igen tangenterna; här behövs bara de enklaste.

3.1 Skriva i VERK

Tryck RETURN vid `Program:`, så att du kommer till BASIC, och skriv

```
RUN MF1:VERK
```

VERK visar sin introduktion. Tryck på en tangent, och du har en tom text framför dig. Skriv programmet, med RETURN efter varje rad och med blanksteg först på raderna som du vill dra in:

```
PROGRAM KUB;
(* Talen 1 till 5 i kub. *)
VAR I: INTEGER;
BEGIN
  FOR I := 1 TO 5 DO
    WRITE(I*I*I:5);
  WRITELN
END.









VERK markören 119 av 119 fritt 15609
```

Det som står mellan (* och *) är en kommentar. Kompilatorn hoppar över den, och den får innehålla å, ä och ö.

Flytta markören med CTRL-S och CTRL-D, CTRL-E och CTRL-X, och ta bort med ←, som i VERK:s bruksanvisning. VERK:s fält och celler behövs inte; tryck inte CTRL-K.

3.2 Spara

Tryck CTRL-Q för menyn, 6 för att spara, och skriv namnet KUB.PAS:

```
Filens namn: KUB.PAS
Sparad: KUB.PAS
Tryck på en tangent.
```

Namnet ska sluta med .PAS; utan det sparar VERK som KUB.VRK. Filen hamnar på skivan i MF0, och det är därför Pascal-skivan ska sitta där. Tryck på en tangent, så kommer menyn tillbaka. 1 fortsätter med texten, och 4 slutar VERK.

Observera. VERK skriver %% på en egen rad sist i filen, efter END.. Där sparar VERK sina celler. Kompilatorn läser inte längre än till END., så det gör ingenting.

3.3 Kompilera och köra

VERK har använt samma minne som Pascals tolk. Välj 4 i menyn och starta om datorn med skivorna kvar, så laddas tolken igen. Kompilera och kör som i kapitel 2:

```
Pascal för ABC 802. PC kompilerar, t.ex. PC.PAS till PC.PK.
Z=CALL(32768) kör ett program igen.

Program: PC
KUB.PAS
ABC802
Z=CALL(32768)
Program: KUB
    1    8   27   64  125
ABC802
```

Sedan kan du köra KUB igen med Z=CALL(32768) så många gånger du vill, utan att starta om.

3.4 När något är fel

Skriv till exempel `WRITLN` i stället för `WRITELN` på den sjunde raden, spara och kompilera. Då slutar kompilatorn vid felet:

```
Pascal för ABC 802. PC kompilerar, t.ex. PC.PAS till PC.PK.
Z=CALL(32768) kör ett program igen.

Program: PC
KUB.PAS
rad 7, kolumn 3: fel 7
ABC802
```

Felet är på rad 7, i kolumn 3, och fel 7 betyder *okänt namn* (se kapitel 6). Raderna räknas från 1, och kommentarer och tomma rader räknas med. Kolumnen är ungefär där kompilatorn märkte felet. Ibland är det i slutet av ordet före: ett glömt semikolon märks först i början av nästa sats.

Kompilatorn slutar vid det första felet och skriver ingen p-kod. Starta VERK igen, välj 7 i menyn och läs in `KUB.PAS`:

```
PROGRAM KUB;
(* Talen 1 till 5 i kub. *)
VAR I: INTEGER;
BEGIN
  FOR I := 1 TO 5 DO
    WRITE(I*I*I:5);
  WRITLN
END.









VERK markören 0 av 118 fritt 15610
```

Rätta raden, spara med 6 och samma namn, och kompilera igen.

3.5 Utan VERK

Ett BASIC-program kan också skriva en Pascalfil, med `PREPARE` och `PRINT #1`. Men BASIC packar två eller flera blanksteg i rad till en kod för tabulering, som kompilatorn inte känner igen (fel 1). Program som skrivs från BASIC får alltså inte ha indrag eller dubbla blanksteg.

Under CP/M finns editorn SKRIV; se kapitel 5.

4. SPRÅKET

Det här kapitlet beskriver det Pascal som PC kompilerar. Det är en delmängd av Wirths Pascal: det mesta som behövs i små och medelstora program finns, och det som saknas står i avsnitt 4.10.

4.1 Programmet

Ett program har ett huvud, deklarationer och en sammansatt sats, och slutar med en punkt:

```
PROGRAM NAMN(INPUT, OUTPUT);
CONST ...
TYPE ...
VAR ...
PROCEDURE ...; FUNCTION ...;
BEGIN
  satser, åtskilda med ;
END.
```

Parenteserna efter namnet får utelämnas. Deklarationerna kommer i den ordningen, och var och en av dem får saknas.

4.2 Tecknen på ABC 802

ABC 802 har å, ä och ö där andra datorer har `[ ] { }` och några tecken till. Därför skrivs Pascal så här på ABC 802:

- Kommentarer skrivs `(* ... *)`. `{ }` är ä och å på skärmen och fungerar inte som kommentar.
- Index i fält skrivs `(. .)`: `A(.I.)` i stället för `A[I]`. Hakparenteserna fungerar också, men de syns som ÄÅ.
- Namn består av bokstäverna A–Z och siffrorna 0–9 och börjar med en bokstav. Å, ä och ö får inte stå i namn, bara i strängar och kommentarer. Små och stora bokstäver är samma. Ett namn får vara 32 tecken långt, och alla tecknen räknas.
- Strängar skrivs med apostrofer, `'Hej'`, och `''` är en apostrof inne i strängen.

4.3 Typerna

INTEGER	heltal från $-32\,768$ till $32\,767$
REAL	reella tal med omkring 9 siffror, från ungefär 10^{-38} till 10^{38}
BOOLEAN	TRUE och FALSE
CHAR	ett tecken
ARRAY	ett fält, till exempel ARRAY (.1..10.) OF INTEGER
TEXT	en textfil, se avsnitt 4.9

Ett fält har en dimension, och gränserna är konstanter. Indexet kontrolleras när programmet körs: utanför gränserna blir körfel 1.

Ett heltal blir reellt där ett reellt tal behövs. Åt andra hållet går det med TRUNC eller ROUND.

En *sträng* är ett PACKED ARRAY (.1..N.) OF CHAR, högst 80 tecken. Strängar kan tilldelas hela, jämföras med = <> < <= > >= och skrivas med WRITE, och en strängkonstant med samma längd kan tilldelas en strängvariabel:

```
TYPE NAMN = PACKED ARRAY (.1..6.) OF CHAR;
VAR A : NAMN;
...
A := 'PASCAL';
IF A < 'ZILOG ' THEN WRITELN(A)
```

4.4 Deklarationerna

```
CONST MAX = 100; PI = 3.14159; JA = 'J';
TYPE TABELL = ARRAY (.1..MAX.) OF INTEGER;
VAR T : TABELL; I, J : INTEGER; X : REAL;
```

TYPE ger namn åt fälttyper, så att samma typ kan användas för flera variabler och parametrar.

Procedurer och funktioner deklareras efter variablerna och har egna konstanter, typer, variabler och procedurer inuti:

```
FUNCTION STORST(VAR T : TABELL; N : INTEGER) : INTEGER;
VAR I, S : INTEGER;
BEGIN
  S := T(.1.);
  FOR I := 2 TO N DO
    IF T(.I.) > S THEN S := T(.I.);
  STORST := S
END;
```

En parameter med VAR framför är en *variabelparameter*: proceduren arbetar med anroparens variabel. Utan VAR får proceduren en kopia av värdet. Fält ska vara variabelparametrar i PC (fel 21 annars). Funktionens värde ges genom att tilldela funktionens namn.

En procedur som anropas innan den är deklarerad skrivs först med bara huvudet och FORWARD, och senare med blocket:

```
PROCEDURE B(N : INTEGER); FORWARD;
PROCEDURE A(N : INTEGER);
BEGIN IF N > 0 THEN B(N - 1) END;
PROCEDURE B;
BEGIN WRITELN(N); A(N) END;
```

4.5 Satserna

`X := uttryck`	tilldelning
`NAMN(a, b)`	anrop av en procedur
`BEGIN s; s END`	flera satser som en
`IF v THEN s ELSE s`	val; ELSE kan utelämnas
`WHILE v DO s`	så länge v är sant
`REPEAT s; s UNTIL v`	tills v blir sant, minst en gång
`FOR I := a TO b DO s`	I från a uppåt till b; DOWNTO räknar nedåt
`CASE x OF ... END`	val efter ett värde

I CASE står ett eller flera värden före kolon:

```
CASE C OF
  'A', 'E', 'I', 'O', 'U', 'Y': WRITELN('vokal');
  '0': WRITELN('noll')
END
```

Finns inget av värdena blir det körfel 4.

4.6 Uttrycken

`* / DIV MOD AND`	multiplikation, division, heltalsdivision, resten, och
`+ - OR`	addition, subtraktion, eller
`= <> < <= > >=`	jämförelser
`NOT`	icke

Det första står högst: `A + B * C` är `A + (B * C)`. Jämförelserna står lägst, så två villkor med `AND` och `OR` skrivs med parenteser: `(I > 0) AND (I < 10)`. `/` ger alltid ett reellt tal, också av två heltal; `DIV` ger heltalet.

4.7 Standardfunktionerna

`ABS(x)` `SQR(x)`	absolutbeloppet, kvadraten, av samma typ
`ODD(i)`	sant om `i` är udda
`ORD(c)` `CHR(i)`	tecknets kod, och tecknet med koden
`SUCC(x)` `PRED(x)`	nästa och föregående värde
`SQRT` `SIN` `COS`	roten, sinus och cosinus
`ARCTAN` `LN` `EXP`	arcus tangens, logaritmen, e^x
`TRUNC(x)` `ROUND(x)`	heltalet, avkortat och avrundat
`EOF` `EOLN`	slutet av filen, av raden

Vinklarna är i radianer. `HALT` avslutar programmet var som helst.

4.8 Skriva och läsa

`WRITE` skriver ett eller flera värden, och `WRITELN` gör dessutom ny rad. Efter ett värde kan bredden stå: `WRITE(I:5)` skriver `I` högerställt i fem tecken. Ett reellt tal kan också få antalet decimaler: `WRITE(X:8:3)`. Utan bredd tar värdet bara de tecken det behöver, och ett reellt tal skrivs som i BASIC. `TRUE` och `FALSE` skrivs med ord.

`READ` läser tal och tecken från tangentbordet, och `READLN` hoppar dessutom till nästa rad. Det du skriver syns på skärmen och lämnas med RETURN.

```
WRITE('Hur många? ');
READLN(N);
```

4.9 Filerna

En variabel av typen `TEXT` är en textfil på skivan. Filens namn står i `RESET` och `REWRITE`:

```
VAR F : TEXT; C : CHAR;
...
REWRITE(F, 'UT.TXT');           (* en ny fil att skriva *)
WRITELN(F, 'första raden');
CLOSE(F);
RESET(F, 'UT.TXT');             (* en fil att läsa *)
```

```
WHILE NOT EOF(F) DO
BEGIN
  WHILE NOT EOLN(F) DO BEGIN READ(F, C); WRITE(C) END;
  READLN(F); WRITELN
END;
CLOSE(F)
```

Namnet kan också vara en strängvariabel; det slutar vid det första blanksteget. Två filer får vara öppna samtidigt. Glöm inte `CLOSE` när du skrivit: det sista av filen skrivs först då.

4.10 Det som inte finns

`PC` har inte `RECORD` och `WITH`, fält av fält, pekare, mängder, uppräknade typer och delomfång. Allt det finns i `PCS` under CP/M (kapitel 5). Ingen av dem har `GOTO`, procedurer som parametrar och variantposter.

5. PASCAL UNDER CP/M

5.1 PVM.COM

Under CP/M är tolken ett vanligt CP/M-program, `PVM.COM`. Sätt CP/M-skivan i A: och Pascal-skivan för CP/M i B:, starta CP/M och gå till B:. `PVM` med ett programnamn kör programmet; `PVM PC` kompilerar:

```
CP/M 2.2 på ABC 802, 64 KB

A>B:
B>DIR
B: HALSA    PAS : LASA     PAS : PC       PAS : PC       PK
B: PCS      PAS : PCS      PK  : PRIMTAL  PAS : PVM      COM
B: REELLA   PAS : REKURSIO PAS : SATSER   PAS : SORTERA  PAS
B: STRANGAR PAS
B>PVM PC
HALSA.PAS

B>PVM HALSA
Hej fran Pascal!
   1   4   9  16  25  36  49  64  81 100
Summan 1..10 = 55

B>
```

Namnet på källan skriver du med stora bokstäver när kompilatorn frågar, som i BASIC-läget. Felen ser likadana ut.

Under CP/M finns hela minnet till programmen: omkring 49 kilobyte i stället för 24. Programmen från BASIC-läget går här som de är, och `.PK`-filerna kan flyttas mellan skivorna.

5.2 Att skriva med SKRIV

CP/M-skivan har editorn SKRIV, med WordStars tangenter:

```
A>SKRIV B:KUB.PAS
```

CTRL-K `X` sparar och går tillbaka till CP/M, och CTRL-K `Q` slutar utan att spara. Filen `SKRIV.TXT` på A: beskriver resten. Med SKRIV slipper du starta om mellan editorn och kompilatorn: skriv, kompilera och kör om och om igen från `B>`.

5.3 Det större språket: PCS

`PVM PCS` är en kompilator för ett större Pascal, och den används likadant. Den kan allt som `PC` kan och dessutom:

Uppräknade typer `FARG = (ROD, GRON, BLA)`. `ORD`, `SUCC`, `PRED`, jämförelser, `FOR` och `CASE` går på dem, och de kan vara index: `ARRAY (.FARG.) OF INTEGER`.

Delomfång `SIFFRA = 0..9`. En tilldelning utanför omfånget ger körfel 1.

Poster `RECORD` med fält, `P.NAMN`, och `WITH P DO`. Poster kan tilldelas hela.

Fält av fält `ARRAY (.1..3, 1..3.) OF INTEGER`, och `M(.I, J.)`.

Fält och poster som värdeparametrar proceduren får en kopia.

Pekare `^T`, `NEW(P)`, `P^`, och `NIL`. `MARK(P)` och `RELEASE(P)` lämnar tillbaka allt minne som `NEW` har tagit efter `MARK`.

Mängder `SET OF CHAR`, `(.'A'..'Z'.)`, `+ - *`, `IN`, och `<= >=` för delmängd. Elementen ska ligga i 0–255.

Ett exempel:

```
TYPE FARG = (ROD, GRON, BLA);
     PERSON = RECORD
       NAMN : PACKED ARRAY (.1..8.) OF CHAR;
       ALDER : INTEGER;
       FARG : FARG
     END;
VAR P : PERSON;
...
WITH P DO
BEGIN
  NAMN := 'ANNA    '; ALDER := 30; FARG := GRON
END
```

Observera. Ett program som använder något av det nya kan bara köras i `PVM.COM`. Tolken i BASIC-läget svarar `inte en p-kodfil`.

`PCS` skriver p-koden först till en fil med typen `.TMP` och gör `.PK`-filen när kompileringen är klar. Kompilatorn är större än `PC` och tar längre tid; för små program märks det inte.

6. FELMEDDELANDEN

6.1 Kompileringsfel

Kompilatorn skriver `rad r, kolumn k: fel n` och slutar. Numret betyder:

1 okänt tecken; till exempel { eller en tabulering från BASIC
2 strängen tar inte slut på raden
3 kommentaren tar inte slut
4 talet är för stort
5 här ska stå något annat: ett `;`, `)`, `THEN`, `END` ...
6 namnet finns redan
7 okänt namn; stavfel, eller inte deklarerat
8 fel typ
9 här ska stå en konstant
10 här ska stå en typ
11 fältets gränser
12 här ska stå en variabel
13 fel antal parametrar
14 här ska stå en sats
15 för många lokala variabler
16 fältet är för stort
17 fallet finns redan, i `CASE`
18 `CASE` har för stort omfång
19 proceduren är `FORWARD` men har inget block
20 här ska stå `PACKED ARRAY OF CHAR`
21 fält som värdeparameter (bara `PC`; skriv `VAR`)
22 funktionens värde kan bara ges i funktionen själv
23 programmet tar inte slut med `END`.
24 strängen är längre än 80 tecken
25 namnet är längre än 32 tecken
26 kompilatorns tabeller är fulla
27 talet har fler än 32 tecken
28 mängdens element ska ligga i 0..255 (`PCS`)

Fel 5 är det vanligaste. Det kommer ofta av ett glömt `;` på raden före, eller ett `;` före `ELSE`, som inte får stå där.

6.2 Körfel

När något går fel medan programmet körs skriver tolken till exempel

```
Pascal för ABC 802. PC kompilerar, t.ex. PC.PAS till PC.PK.
Z=CALL(32768) kör ett program igen.

Program: PC
INDEX.PAS
ABC802
Z=CALL(32768)
Program: INDEX

KÖRFEL 1: index utanför gränserna, PC 901D
ABC802
```

PC är platsen i p-koden. Numret betyder:

1 index utanför gränserna, eller ett delomfång
2 division med noll
3 stacken är full; oftast en procedur som anropar sig själv utan slut
4 CASE utan fall för värdet
5 filen: den finns inte, skivan är full, eller fler än två filer
6 okänd instruktion; .PK-filen är skadad
7 okänd standardprocedur
8 flyttalsfel, till exempel roten ur ett negativt tal
9 minnet är fullt (NEW, under CP/M)

6.3 Övriga

`filen finns inte`	det finns ingen .PK-fil med det namnet
`inte en p-kodfil`	filen är inte p-kod, eller från PCS
`programmet ryms inte`	p-koden och variablerna får inte plats

```
Pascal för ABC 802. PC kompilerar, t.ex. PC.PAS till PC.PK.
Z=CALL(32768) kör ett program igen.

Program: NOLL
filen finns inte
ABC802
```

7. HUR PASCAL FUNGERAR

Det här kapitlet är för den som vill veta vad som händer inne i datorn. Du behöver det inte för att skriva program.

7.1 Kompilatorn

PC läser källan en gång, från början till slut, och skriver p-koden under tiden. Det kallas en kompilator i *ett pass*. Därför måste allt deklareras innan det används, och därför finns FORWARD. Varje procedur samlas i en buffert och skrivs till filen när den är klar.

Kompilatorn är 13 432 bytes p-kod. Den är skriven i samma delmängd som den kompilerar, och kompilerad med sig själv blir den byte för byte densamma. Att kompilera PC.PAS, omkring 1 600 rader, tar ungefär tolv minuter i BASIC-läget.

7.2 P-koden

P-koden är instruktioner för en stackmaskin. En instruktion lägger ett värde på stacken, en annan räknar med de två översta, en tredje sparar resultatet i en variabel:

```
I := I + 1        LOD I   LDC 1   ADI   STR I
```

Varje instruktion är en byte, ofta med en eller två bytes till. De vanligaste har korta former, så att ett genomsnitt blir under tre bytes. Det gör att PC får plats: samma program i Z80-kod skulle bli flera gånger större.

.PK-filen börjar med PK och ett versionsnummer. Version 1 går i båda tolkarna, och version 2, med det nya från PCS, bara i PVM.COM.

7.3 Tolken

Tolken är 4 089 bytes Z80-kod. Den läser en p-instruktion, hoppar via en tabell till rutinen för den och tar nästa. Skärmen, tangentbordet och skivan nås genom ABC 802:s ROM, med samma rutiner som BASIC använder. Reella tal räknas med FPP.BIN, räknerutinerna ur BBC BASIC, som laddas först när ett program behöver dem.

Under BASIC ligger tolken på adress 32768, och p-koden och variablerna efter den, upp till där BASIC:s skivbuffertar börjar. Därför är det `Z=CALL(32768)` som startar den igen.

Under CP/M är `PVM.COM` samma tolk, med CP/M:s anrop i stället för ROM:ens och med räknerutinerna i samma fil. Kompilatorn kompilerar sig själv på ungefär samma tid som i BASIC-läget, och `PCS` på omkring tjugo minuter.

Mer om Pascal finns i `pascal/PASCAL.md`.

8. MINNESLISTA

BASIC-läget

MF0, MF1	Pascal-skivan, programskivan
`Program: PC`	kompilera; sedan källans namn
`Program: HALSA`	köra `HALSA.PK`
`Z=CALL(32768)`	tolken igen, från BASIC
`RUN MF1:VERK`	skriva programmet
CTRL-Q `6, 7, 4`	spara, läsa, sluta VERK

efter VERK: starta om datorn

CP/M

A:, B:	CP/M-skivan, Pascal-skivan
`SKRIV B:KUB.PAS`	skriva programmet
`PVM PC`, `PVM PCS`	kompilera
`PVM KUB`	köra `KUB.PK`

På ABC 802

`(* *)`	kommentar
`A(.I.)`	index
`'Hej'`	sträng, högst 80 tecken

Felen

`rad 7, kolumn 3: fel 7`	kompileringsfel, se 6.1
`KÖRFEL 1: …`	körfel, se 6.2